IPSSI — 2e année Bachelor Informatique

Mathématiques appliquées à l'Intelligence Artificielle

Guide Google Colab

Prise en main de l'environnement Python pour le module

Mohamed EL AFRIT — www.mohamedelafrit.com

Année académique 2025-2026

CC BY-NC-SA 4.0

Table des matières

Table des matières 2
1. Introduction 3
1.1 Qu'est-ce que Google Colab ? 3
1.2 Pourquoi Colab dans ce module ? 3
2. Premiers pas 3
2.1 Créer un compte Google 3
2.2 Accéder à Google Colab 3
2.3 Créer un nouveau notebook 3
2.4 L'interface 4
3. Les cellules 4
3.1 Cellule de code 4
3.2 Cellule de texte (Markdown) 4
3.3 Ajouter, supprimer, déplacer 4
3.4 Raccourcis clavier essentiels 4
4. Exécuter du code 5
4.1 Le bouton Play 5
4.2 L'ordre d'exécution 5
4.3 Le runtime (environnement d'exécution) 5
5. Bibliothèques Python 5
5.1 Bibliothèques déjà installées 5
5.2 Installer un package supplémentaire 6
5.3 Importer une bibliothèque 6
6. Gérer ses fichiers 6
6.1 Le système de fichiers Colab 6
6.2 Monter Google Drive 7
6.3 Importer/exporter des fichiers 7
7. Astuces pour le module 7
7.1 Écrire des mathématiques en Markdown 7
7.2 Afficher des graphiques 7
7.3 Bonnes pratiques 7
8. Résolution de problèmes 8
8.1 « Mon code ne s'exécute pas » 8
8.2 « NameError: variable non définie » 8
8.3 « Le graphique ne s'affiche pas » 8
8.4 Le runtime est lent ou bloqué 8

© 2025 Mohamed EL AFRIT — www.mohamedelafrit.com — CC BY-NC-SA 4.0 — 2025-2026

1. Introduction

1.1 Qu'est-ce que Google Colab ?

Google Colaboratory (Colab) est un service gratuit de Google qui permet d'écrire et d'exécuter du code Python directement dans un navigateur web, sans rien installer sur votre ordinateur.

Colab fournit un environnement complet avec des bibliothèques de science des données déjà installées (NumPy, Matplotlib, scikit-learn, TensorFlow) et même un accès gratuit à des GPU pour accélérer les calculs.

1.2 Pourquoi Colab dans ce module ?

- Gratuit : aucun logiciel payant, juste un compte Google
- Zéro configuration : Python, NumPy, Matplotlib sont déjà prêts
- Accessible partout : il suffit d'un navigateur et d'une connexion internet
- Collaboration : vous pouvez partager vos notebooks comme un Google Doc
- Sauvegarde automatique dans Google Drive

> **✓ Astuce**
> Même si vous avez Python installé en local (Anaconda, VS Code…), utilisez Colab pour les TP de ce module afin que tout le monde travaille dans le même environnement.

2. Premiers pas

2.1 Créer un compte Google

Si vous n'avez pas de compte Google, rendez-vous sur accounts.google.com et créez-en un gratuitement. Votre compte IPSSI peut fonctionner s'il est lié à Google Workspace.

2.2 Accéder à Google Colab

Ouvrez votre navigateur et allez sur : **colab.research.google.com**

Vous arriverez sur la page d'accueil de Colab avec vos notebooks récents et des exemples.

2.3 Créer un nouveau notebook

1. Cliquez sur Fichier > Nouveau notebook (ou le bouton « + Nouveau notebook »)
2. Un notebook vierge s'ouvre avec une cellule de code prête à l'emploi
3. Renommez-le en cliquant sur « Untitled0.ipynb » en haut à gauche

> **✓ Astuce**
> Nommez vos notebooks de manière explicite : par exemple « Jour1_Regression.ipynb » ou « Jour3_Perceptron.ipynb ».

© 2025 Mohamed EL AFRIT — www.mohamedelafrit.com — CC BY-NC-SA 4.0 — 2025-2026

2.4 L'interface

L'interface de Colab est composée de :

- Barre de menu : Fichier, Édition, Affichage, Insertion, Exécution, Outils, Aide
- Barre d'outils : boutons pour ajouter des cellules, exécuter, etc.
- Zone de cellules : l'espace principal où vous écrivez votre code et vos explications
- Panneau latéral (icônes à gauche) : fichiers, variables, secrets

3. Les cellules

3.1 Cellule de code

C'est le type de cellule par défaut. Vous y écrivez du code Python qui sera exécuté lorsque vous cliquerez sur le bouton Play ou appuierez sur Ctrl+Entrée.

```
# Ceci est une cellule de code Python
import numpy as np
x = np.array([1, 2, 3, 4, 5])
print(f"Somme = {np.sum(x)}")
```

3.2 Cellule de texte (Markdown)

Les cellules de texte utilisent le format Markdown pour écrire des explications, des titres, des listes, et même des formules mathématiques.

Pour ajouter une cellule texte : cliquez sur + Texte dans la barre d'outils.

```
# Titre principal
## Sous-titre
**texte en gras**, *texte en italique*
$E = mc^2$ (formule en ligne)
$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$ (formule centrée)
```

3.3 Ajouter, supprimer, déplacer

- Ajouter : boutons + Code et + Texte en haut, ou entre deux cellules
- Supprimer : icône poubelle en haut à droite de la cellule, ou Édition > Supprimer
- Déplacer : flèches haut/bas en haut à droite, ou glisser-déposer

3.4 Raccourcis clavier essentiels

Raccourci	Action
`Ctrl + Entrée`	Exécuter la cellule active
`Shift + Entrée`	Exécuter et passer à la suivante
`Ctrl + M, B`	Insérer une cellule en dessous

© 2025 Mohamed EL AFRIT — www.mohamedelafrit.com — CC BY-NC-SA 4.0 — 2025-2026

`Ctrl + M, A`	Insérer une cellule au-dessus
`Ctrl + M, D`	Supprimer la cellule active
`Ctrl + M, Y`	Convertir en cellule de code
`Ctrl + M, M`	Convertir en cellule Markdown
`Ctrl + S`	Sauvegarder le notebook
`Ctrl + Z`	Annuler

4. Exécuter du code

4.1 Le bouton Play

Chaque cellule de code a un bouton Play (▶) à sa gauche. Cliquez dessus pour exécuter le code. Un numéro entre crochets [1] apparaît, indiquant l'ordre d'exécution.

4.2 L'ordre d'exécution

⚠ Attention
Piège fréquent ! Les cellules ne s'exécutent PAS automatiquement de haut en bas. Elles s'exécutent dans l'ordre où vous cliquez sur Play. Si vous exécutez la cellule 3 avant la cellule 1, les variables définies dans la cellule 1 ne seront pas disponibles et vous obtiendrez une erreur NameError.

✓ Astuce
En cas de doute, utilisez Exécution > Tout exécuter (ou Ctrl+F9) pour relancer toutes les cellules dans l'ordre, du début à la fin.

4.3 Le runtime (environnement d'exécution)

Le runtime est la machine virtuelle qui exécute votre code. Quand vous ouvrez un notebook, Colab vous connecte automatiquement à un runtime.

- Connexion : automatique quand vous exécutez une cellule
- Déconnexion : après 90 min d'inactivité ou 12h de session continue
- Redirection : Exécution > Redémarrer le runtime (remet tout à zéro)

⚠ Attention
Quand le runtime se déconnecte, toutes vos variables et données en mémoire sont perdues. Votre code (le texte des cellules) est sauvegardé, mais il faut tout ré-exécuter.

5. Bibliothèques Python

5.1 Bibliothèques déjà installées

Colab fournit un environnement Python riche. Les bibliothèques utilisées dans ce module sont toutes pré-installées :

Raccourci	Action
`numpy`	Calcul numérique (vecteurs, matrices)
`matplotlib`	Visualisation (graphiques, courbes)
`scikit-learn`	Machine learning (régression, classification)
`tensorflow / keras`	Réseaux de neurones (Jour 4)
`pandas`	Manipulation de données (optionnel)

5.2 Installer un package supplémentaire

Si vous avez besoin d'un package non installé, utilisez la commande pip avec un point d'exclamation :

```
!pip install nom_du_package
```

Exemple :

```
!pip install seaborn
```

> ⚠ **Attention**
> Les packages installés manuellement ne persistent pas après une déconnexion du runtime. Il faudra les réinstaller.

5.3 Importer une bibliothèque

Au début de chaque notebook, importez les bibliothèques nécessaires :

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

> ✓ **Astuce**
> Mettez tous vos imports dans la première cellule du notebook. C'est une bonne pratique qui facilite la lecture et la maintenance.

6. Gérer ses fichiers

6.1 Le système de fichiers Colab

Colab fournit un espace de stockage temporaire accessible via le panneau Fichiers (à gauche). Cet espace est supprimé quand le runtime se déconnecte.

> ⚠ **Attention**
> Ne stockez RIEN d'important uniquement dans le système de fichiers Colab. Utilisez Google Drive pour sauvegarder vos données.

6.2 Monter Google Drive

Pour accéder à vos fichiers Google Drive depuis Colab :

```
from google.colab import drive
drive.mount('/content/drive')
```

Colab vous demandera d'autoriser l'accès. Après confirmation, vos fichiers Drive seront accessibles dans /content/drive/MyDrive/.

6.3 Importer/exporter des fichiers

Importer un fichier depuis votre ordinateur :

```
from google.colab import files
uploaded = files.upload()  # ouvre un dialogue
```

Télécharger un fichier vers votre ordinateur :

```
files.download('mon_fichier.csv')
```

7. Astuces pour le module

7.1 Écrire des mathématiques en Markdown

Dans une cellule Markdown, vous pouvez écrire des formules LaTeX :

- Formule en ligne : $formule$ (avec un seul dollar de chaque côté)
- Formule centrée : $$formule$$ (avec deux dollars)

Exemples :

```
$\hat{y} = \mathbf{w}^T \mathbf{x} + b$
$$J(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$
```

7.2 Afficher des graphiques

Matplotlib affiche les graphiques directement sous la cellule de code. Pas besoin de commande spéciale dans Colab (contrairement à Jupyter classique où il faut %matplotlib inline).

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [1, 4, 9])
plt.title('Mon graphique')
plt.show()  # optionnel dans Colab, mais bonne pratique
```

7.3 Bonnes pratiques

4. Commentez votre code en français (# ceci est un commentaire)
5. Faites des cellules courtes (5-15 lignes max par cellule)
6. Exécutez toujours les cellules dans l'ordre (de haut en bas)

© 2025 Mohamed EL AFRIT — www.mohamedelafrit.com — CC BY-NC-SA 4.0 — 2025-2026

7. Ajoutez des cellules Markdown pour expliquer votre démarche
8. Sauvegardez régulièrement (Ctrl+S, mais Colab sauvegarde aussi automatiquement)

> ✓ **Astuce**
> Prenez l'habitude de redémarrer le runtime et tout ré-exécuter avant de rendre un TP. Cela garantit que votre notebook fonctionne de A à Z sans erreur.

8. Résolution de problèmes

Voici les problèmes les plus fréquents et leurs solutions :

8.1 « Mon code ne s'exécute pas »

- Vérifiez que le runtime est connecté (indicateur vert en haut à droite)
- Si le runtime est déconnecté, cliquez sur Connecter ou exécutez une cellule
- Si la cellule affiche un sablier, elle est en attente : patientez ou interrompez

8.2 « NameError: variable non définie »

> ⚠ **Attention**
> C'est le problème le plus courant ! Il signifie que vous utilisez une variable qui n'a pas encore été créée dans ce runtime. Solution : exécutez les cellules précédentes dans l'ordre, ou utilisez Exécution > Tout exécuter.

8.3 « Le graphique ne s'affiche pas »

- Ajoutez plt.show() à la fin de votre code de graphique
- Vérifiez que vous avez bien importé matplotlib : import matplotlib.pyplot as plt
- Si le graphique est vide, vérifiez vos données (tableaux vides ou NaN)

8.4 Le runtime est lent ou bloqué

- Allez dans Exécution > Interrompre l'exécution pour arrêter une cellule en cours
- Puis Exécution > Redémarrer le runtime pour repartir de zéro
- Ré-exécutez toutes les cellules avec Exécution > Tout exécuter

> ✓ **Astuce**
> En cas de problème persistant, la solution universelle est : 1) Redémarrer le runtime, 2) Tout ré-exécuter. Cela résout 90% des problèmes.

© 2025 Mohamed EL AFRIT — IPSSI — www.mohamedelafrit.com — CC BY-NC-SA 4.0